

SmartShop AI API Endpoint Architecture

Project: AI-Powered Smart E-commerce Platform

Frontend: React.js customer, admin, and guest experience

Backend target: ASP.NET Core Web API, SQL Server, Entity Framework Core

Recommended base URL: /api/v1

This document lists the backend endpoints needed to replace the current localStorage mock workflow with a real ASP.NET Core and SQL Server API. Blog/news endpoints are intentionally not included because that feature was removed from the frontend.

1. Architecture Overview

React Frontend

- Guest shop
- Customer account, cart, checkout, recommendations
- Admin dashboard

ASP.NET Core Web API

- Controllers
- DTO validation
- JWT authentication and role authorization
- Application services

Application Layer

- AuthService
- ProductService
- CartService
- OrderService
- RecommendationService
- AnalyticsService
- ForecastingService
- AdminService

Infrastructure Layer

- Entity Framework Core
- SQL Server
- Optional cache for catalog, recommendations, and dashboard KPIs
- Optional background jobs for forecast/recommendation refresh

2. API Conventions

Authentication

Use JWT bearer authentication.

Roles:

- Guest: no token
- User: registered customer
- Admin: admin user, initially seeded as admin@shop.com

Standard Response Envelope

```
{
  "success": true,
  "message": "Operation completed",
  "data": {},
  "errors": []
}
```

Pagination Query Convention

SmartShop AI API Endpoint Architecture

Use this for list endpoints:

?page=1&pageSize=12&search=headphone&sortBy=price&sortOrder=asc

Response metadata:

```
{
  "items": [],
  "page": 1,
  "pageSize": 12,
  "totalItems": 100,
  "totalPages": 9
}
```

Common HTTP Status Codes

- 200 OK: read/update successful
- 201 Created: create successful
- 204 No Content: delete successful
- 400 Bad Request: validation error
- 401 Unauthorized: missing/invalid token
- 403 Forbidden: wrong role
- 404 Not Found: resource missing
- 409 Conflict: duplicate email, invalid stock, duplicate review
- 500 Internal Server Error: unexpected server issue

3. Authentication Endpoints

Public

- POST /api/v1/auth/register
 - Registers a regular customer.
 - Body: name, email, password, confirmPassword, address, phoneNumber.
- POST /api/v1/auth/login
 - Logs in customer or admin.
 - Body: email, password, rememberMe.
 - Returns: accessToken, refreshToken, user profile, role.
- POST /api/v1/auth/refresh-token
 - Issues a new access token.
 - Body: refreshToken.
- POST /api/v1/auth/forgot-password
 - Starts mock or real reset flow.
 - Body: email.
- POST /api/v1/auth/reset-password
 - Completes reset flow.
 - Body: email, token, newPassword.

Authenticated

- GET /api/v1/auth/me
 - Returns current logged-in user.
- POST /api/v1/auth/logout
 - Invalidates refresh token.
- PUT /api/v1/auth/password
 - Changes password.

- Body: currentPassword, newPassword.

4. Product Catalog Endpoints

Public Catalog

- GET /api/v1/products
 - Used by home page product grid.
 - Query: search, categoryId, brandId, minPrice, maxPrice, minRating, sortBy, sortOrder, page, pageSize.
- GET /api/v1/products/autocomplete
 - Used by search suggestions.
 - Query: query, limit.
- GET /api/v1/products/{productId}
 - Used by product detail page.
- GET /api/v1/products/{productId}/gallery
 - Product image gallery.
- GET /api/v1/products/{productId}/rating-summary
 - Average rating and star breakdown.
- GET /api/v1/products/compare
 - Used by compare page.
 - Query: ids=p-1,p-2,p-3,p-4.
- GET /api/v1/categories
 - Category filter list.
- GET /api/v1/brands
 - Brand/seller listing page.
- GET /api/v1/brands/{brandId}
 - Brand detail page.
- GET /api/v1/brands/{brandId}/products
 - Products under a brand.

Authenticated Customer

- POST /api/v1/brands/{brandId}/follow
 - Follow a brand.
- DELETE /api/v1/brands/{brandId}/follow
 - Unfollow a brand.

5. Product Reviews Endpoints

Public

- GET /api/v1/products/{productId}/reviews
 - Product review list.
 - Query: page, pageSize, sortBy.

Authenticated Customer

- POST /api/v1/products/{productId}/reviews
 - Creates review.
 - Body: rating, comment.
- PUT /api/v1/reviews/{reviewId}

- Updates customer review.
- DELETE /api/v1/reviews/{reviewId}
- Deletes customer review.

6. Cart Endpoints

All cart endpoints require User role.

- GET /api/v1/cart
- Current user's cart.
- POST /api/v1/cart/items
- Adds item to cart.
- Body: productId, quantity.
- PUT /api/v1/cart/items/{productId}
- Updates item quantity.
- Body: quantity.
- DELETE /api/v1/cart/items/{productId}
- Removes item from cart.
- DELETE /api/v1/cart
- Clears cart.
- POST /api/v1/cart/items/{productId}/save-for-later
- Moves cart item to wishlist.

7. Wishlist Endpoints

All wishlist endpoints require User role.

- GET /api/v1/wishlist
- Current user's wishlist.
- POST /api/v1/wishlist/items/{productId}
- Adds product to wishlist.
- DELETE /api/v1/wishlist/items/{productId}
- Removes product from wishlist.
- POST /api/v1/wishlist/items/{productId}/move-to-cart
- Moves wishlist item to cart.
- Body: quantity.

8. Checkout and Order Endpoints

All checkout/order customer endpoints require User role.

- POST /api/v1/checkout/quote
- Calculates subtotal, shipping, tax, discount, and total.
- Body: cartItems, addressId or address, couponCode.
- POST /api/v1/coupons/validate
- Validates coupon code.
- Body: couponCode, subtotal.
- POST /api/v1/orders
- Places order and clears cart.
- Body: addressId or shippingAddress, paymentMethod, couponCode.
- GET /api/v1/orders/my
- Current user's order history.

- Query: status, page, pageSize.
- GET /api/v1/orders/{orderId}
- Current user's order details.
- GET /api/v1/orders/{orderId}/tracking
- Order tracking timeline.
- POST /api/v1/orders/{orderId}/cancel
- Cancels eligible order.
- POST /api/v1/orders/{orderId}/reorder
- Adds previous order items back to cart.

9. User Profile and Address Endpoints

All endpoints require User role, except admin user management endpoints listed later.

- GET /api/v1/users/me
- Profile overview.
- PUT /api/v1/users/me
- Updates name, email, phone.
- GET /api/v1/users/me/stats
- Account stats for profile overview.
- GET /api/v1/users/me/addresses
- Address book.
- POST /api/v1/users/me/addresses
- Adds address.
- PUT /api/v1/users/me/addresses/{addressId}
- Updates address.
- DELETE /api/v1/users/me/addresses/{addressId}
- Deletes address.

10. Recently Viewed and User Event Endpoints

These endpoints support personalization and recommendation signals.

Guest or User

- POST /api/v1/events/product-view
- Stores product view event.
- Body: productId, anonymousSessionId optional.
- POST /api/v1/events/search
- Stores search event.
- Body: query, filters.
- POST /api/v1/events/recommendation-click
- Stores recommendation click.
- Body: productId, recommendationId, placement.

Authenticated Customer

- GET /api/v1/recently-viewed
- Current user's recent product views.
- POST /api/v1/recently-viewed/{productId}
- Adds product to recently viewed list.

- DELETE /api/v1/recently-viewed
- Clears recently viewed list.

11. AI Recommendation Endpoints

All recommendation hub endpoints require User role.

- GET /api/v1/recommendations
- Returns recommendation hub data.
- Sections: justForYou, trendingNow, seasonalPicks, completeTheLook, recentlyViewed.
- POST /api/v1/recommendations/refresh
- Refreshes recommendation results.
- GET /api/v1/products/{productId}/recommendations
- Related products for product detail page.
- GET /api/v1/recommendations/explanations
- Explanation badge text.
- Query: productId, sourceProductId optional.
- GET /api/v1/trending
- Trending products.
- Query: categoryId optional, limit.

12. Deals and Coupon Endpoints

Public

- GET /api/v1/deals
- Deals page.
- Query: type=today|lightning|clearance, categoryId, page, pageSize.
- GET /api/v1/deals/flash-sale
- Flash sale countdown and active deal metadata.
- POST /api/v1/deals/subscribe
- Email alert signup.
- Body: email.

Admin

- POST /api/v1/admin/deals
- Creates deal.
- PUT /api/v1/admin/deals/{dealId}
- Updates deal.
- DELETE /api/v1/admin/deals/{dealId}
- Deletes deal.

13. Support Center Endpoints

Public

- GET /api/v1/support/faqs
- FAQ accordion data.
- GET /api/v1/support/policies/shipping
- Shipping information.
- GET /api/v1/support/policies/returns

- Returns/refunds policy.
- POST /api/v1/support/tickets
- Contact form submission.
- Body: name, email, orderId optional, message.

Authenticated Customer

- GET /api/v1/support/tickets/my
- Customer's support requests.
- POST /api/v1/support/chat/messages
- Optional live chat placeholder.

14. Admin Product Management Endpoints

All admin endpoints require Admin role.

- GET /api/v1/admin/products
- Admin product table.
- Query: search, categoryId, status, page, pageSize.
- POST /api/v1/admin/products
- Adds product.
- PUT /api/v1/admin/products/{productId}
- Edits product.
- PATCH /api/v1/admin/products/{productId}/stock
- Updates stock.
- PATCH /api/v1/admin/products/{productId}/status
- Activates/deactivates product.
- DELETE /api/v1/admin/products/{productId}
- Deletes product.
- POST /api/v1/admin/products/import
- Bulk import.
- GET /api/v1/admin/products/export
- Bulk export CSV/Excel.

15. Admin Order Management Endpoints

- GET /api/v1/admin/orders
- Admin order table.
- Query: status, paymentMethod, fromDate, toDate, search, page, pageSize.
- GET /api/v1/admin/orders/{orderId}
- Admin order details.
- PATCH /api/v1/admin/orders/{orderId}/status
- Updates order status.
- Body: status.
- PATCH /api/v1/admin/orders/{orderId}/tracking
- Updates tracking note or current location.

16. Admin User Management Endpoints

- GET /api/v1/admin/users
- User list.

SmartShop AI API Endpoint Architecture

- Query: search, role, blocked, page, pageSize.
- GET /api/v1/admin/users/{userId}
- User details.
- PATCH /api/v1/admin/users/{userId}/block
- Blocks user.
- PATCH /api/v1/admin/users/{userId}/unblock
- Unblocks user.
- PATCH /api/v1/admin/users/{userId}/role
- Promotes or demotes user.
- Body: role.
- DELETE /api/v1/admin/users/{userId}
- Deletes user account.

17. Admin Dashboard, Analytics, and Forecasting Endpoints

- GET /api/v1/admin/dashboard/kpis
- Total revenue, total orders, total users, low stock count.
- GET /api/v1/admin/dashboard/recent-orders
- Recent orders widget.
- GET /api/v1/admin/dashboard/top-products
- Top selling products table.
- GET /api/v1/admin/dashboard/low-stock
- Low stock alerts.
- GET /api/v1/admin/analytics/sales
- Sales chart.
- Query: range=7d|30d|90d, groupBy=day|week|month.
- GET /api/v1/admin/forecasting/sales
- Sales forecast with confidence intervals.
- Query: months=3.
- GET /api/v1/admin/forecasting/demand-by-category
- Demand forecast bar chart.
- GET /api/v1/admin/analytics/seasonal-trends
- Seasonal trend analysis.
- GET /api/v1/admin/ai/model-metrics
- Recommendation accuracy, CTR, conversion rate, forecast error, MAPE.
- POST /api/v1/admin/ai/retrain
- Starts AI model retraining job.
- GET /api/v1/admin/ai/retrain/jobs/{jobId}
- Retraining job status.

18. Admin Settings, Backup, and Restore Endpoints

- GET /api/v1/admin/settings/store
- Store name, email, currency, tax rate.
- PUT /api/v1/admin/settings/store
- Updates store settings.
- GET /api/v1/admin/settings/ai
- Recommendation and forecasting toggles, retrain schedule.

- PUT /api/v1/admin/settings/ai
- Updates AI settings.
- POST /api/v1/admin/backup
- Creates backup.
- POST /api/v1/admin/restore
- Restores backup.

19. Frontend Page to API Mapping

Public Guest Pages

- Home page:
- GET /products
- GET /products/autocomplete
- GET /categories
- POST /events/product-view
- Product detail:
- GET /products/{productId}
- GET /products/{productId}/reviews
- GET /products/{productId}/rating-summary
- GET /products/{productId}/recommendations
- Deals:
- GET /deals
- GET /deals/flash-sale
- POST /deals/subscribe
- Brands:
- GET /brands
- GET /brands/{brandId}
- GET /brands/{brandId}/products
- Compare:
- GET /products/compare
- Support:
- GET /support/faqs
- POST /support/tickets

Customer Pages

- Cart:
- GET /cart
- POST /cart/items
- PUT /cart/items/{productId}
- DELETE /cart/items/{productId}
- POST /cart/items/{productId}/save-for-later
- Checkout:
- POST /checkout/quote
- POST /coupons/validate
- POST /orders
- Profile:
- GET /users/me
- PUT /users/me
- GET /orders/my

SmartShop AI API Endpoint Architecture

- GET /wishlist
- GET /users/me/addresses
- Recommendations:
 - GET /recommendations
 - POST /recommendations/refresh
- Order tracking:
 - GET /orders/{orderId}
 - GET /orders/{orderId}/tracking

Admin Pages

- Admin dashboard:
 - GET /admin/dashboard/kpis
 - GET /admin/analytics/sales
 - GET /admin/dashboard/top-products
 - GET /admin/dashboard/recent-orders
 - GET /admin/dashboard/low-stock
- Products tab:
 - GET /admin/products
 - POST /admin/products
 - PUT /admin/products/{productId}
 - DELETE /admin/products/{productId}
- Orders tab:
 - GET /admin/orders
 - GET /admin/orders/{orderId}
 - PATCH /admin/orders/{orderId}/status
- Users tab:
 - GET /admin/users
 - PATCH /admin/users/{userId}/block
 - PATCH /admin/users/{userId}/unblock
 - PATCH /admin/users/{userId}/role
 - DELETE /admin/users/{userId}
- Analytics tab:
 - GET /admin/forecasting/sales
 - GET /admin/forecasting/demand-by-category
 - GET /admin/analytics/seasonal-trends
 - GET /admin/ai/model-metrics
- Settings tab:
 - GET /admin/settings/store
 - PUT /admin/settings/store
 - GET /admin/settings/ai
 - PUT /admin/settings/ai
 - POST /admin/backup
 - POST /admin/restore

20. Recommended SQL Server Entities

- Users
- Roles
- UserRefreshTokens
- Addresses
- Products

- ProductImages
- Categories
- Brands
- BrandFollowers
- Reviews
- Carts
- CartItems
- Wishlists
- WishlistItems
- Orders
- OrderItems
- OrderStatusHistory
- Coupons
- Deals
- DealSubscriptions
- ProductViewEvents
- SearchEvents
- RecommendationEvents
- RecommendationResults
- SupportTickets
- StoreSettings
- AiSettings
- AiModelMetrics
- ForecastResults
- Backups

21. Recommended ASP.NET Core Folder Structure

```
SmartShop.Api
Controllers
AuthController.cs
ProductsController.cs
CategoriesController.cs
BrandsController.cs
ReviewsController.cs
CartController.cs
WishlistController.cs
CheckoutController.cs
OrdersController.cs
RecommendationsController.cs
DealsController.cs
SupportController.cs
Admin
AdminProductsController.cs
AdminOrdersController.cs
AdminUsersController.cs
AdminDashboardController.cs
AdminAnalyticsController.cs
AdminSettingsController.cs
Application
DTOs
Interfaces
Services
Domain
Entities
Enums
Infrastructure
Data
Repositories
Identity
BackgroundJobs
```

22. Suggested Build Priority

1. Auth and users
2. Products, categories, brands, reviews
3. Cart, wishlist, checkout, orders
4. Admin products, orders, users
5. Dashboard KPIs and analytics
6. Recommendations and event tracking
7. Forecasting and AI model metrics
8. Support tickets, deals, backup/restore

23. Minimal First Backend Milestone

If you want the frontend connected quickly, build these first:

- POST /auth/login
- POST /auth/register
- GET /auth/me
- GET /products
- GET /products/{productId}
- GET /categories
- GET /brands
- GET /cart
- POST /cart/items
- PUT /cart/items/{productId}
- DELETE /cart/items/{productId}
- GET /wishlist
- POST /wishlist/items/{productId}
- POST /orders
- GET /orders/my
- GET /admin/dashboard/kpis
- GET /admin/products
- GET /admin/orders
- GET /admin/users